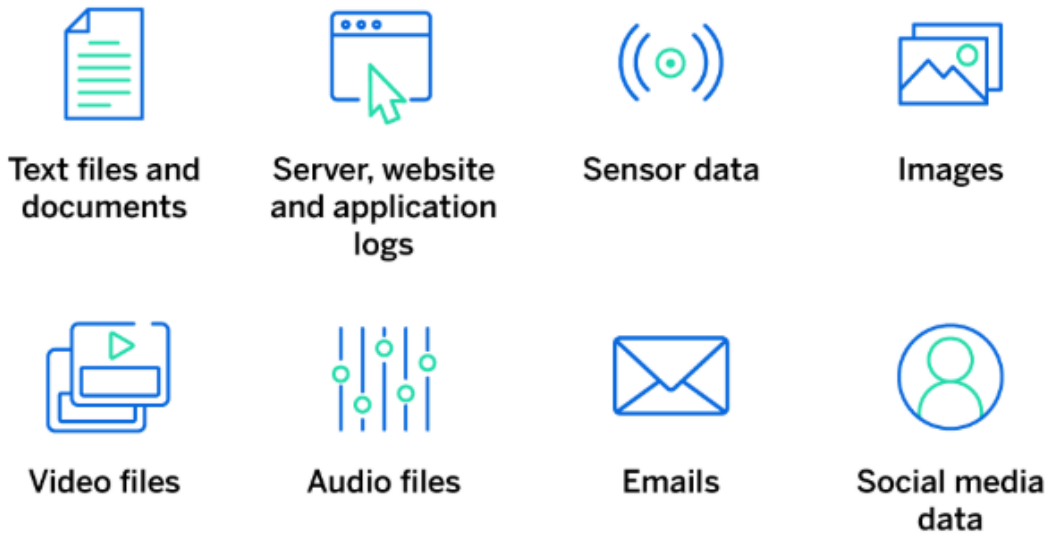
AI-102 Natural Language Processing (NLP)

Dr. Jimmy Ardiansyah

EMERGING & COLLABORATIVE STUDIES

Understanding Text Data

Nature of Text Data



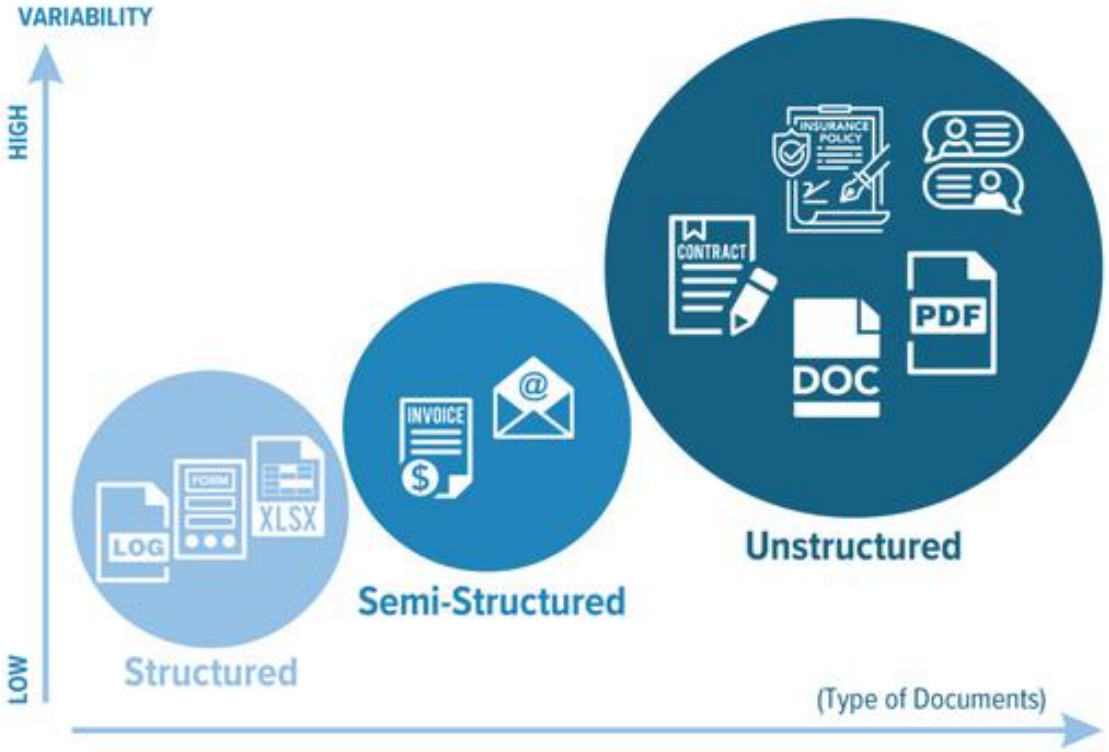
Text data, found in forms such as articles, social media posts, emails, chat messages, and reviews, is inherently unstructured. Unlike numerical data, it lacks an immediate, easy-to-analyze format for machines. The inherent complexity of human language is a primary challenge, encompassing:

Nature of Text Data

Text data, found in forms such as articles, social media posts, emails, chat messages, and reviews, is inherently unstructured. Unlike numerical data, it lacks an immediate, easy-to-analyze format for machines. The inherent complexity of human language is a primary challenge, encompassing:

Ambiguity and Nuances: Words and phrases can have multiple meanings depending on the context, making them difficult to interpret.

Nature of Text Data



Text data, found in forms such as articles, social media posts, emails, chat messages, and reviews, is inherently unstructured. Unlike numerical data, it lacks an immediate, easy-to-analyze format for machines. The inherent complexity of human language is a primary challenge, encompassing:

Ambiguity and Nuances: Words and phrases can have multiple meanings depending on the context, making them difficult to interpret.

Variability: Text varies widely in length, structure, and style, requiring extensive cleaning and standardization.

Nature of Text Data

Text data, found in forms such as articles, social media posts, emails, chat messages, and reviews, is inherently **unstructured**. Unlike numerical data, it lacks an immediate, easy-to-analyze format for machines. The inherent complexity of human language is a primary challenge, encompassing:

Ambiguity and Nuances: Words and phrases can have multiple meanings depending on the context, making them difficult to interpret.

Variability: Text varies widely in length, structure, and style, requiring extensive cleaning and standardization.

Noisy Data: Text often contains irrelevant information like punctuation, common words, and HTML tags that must be removed.

Nature of Text Data

Text data, found in forms such as articles, social media posts, emails, chat messages, and reviews, is inherently **unstructured**. Unlike numerical data, it lacks an immediate, easy-to-analyze format for machines. The inherent complexity of human language is a primary challenge, encompassing:

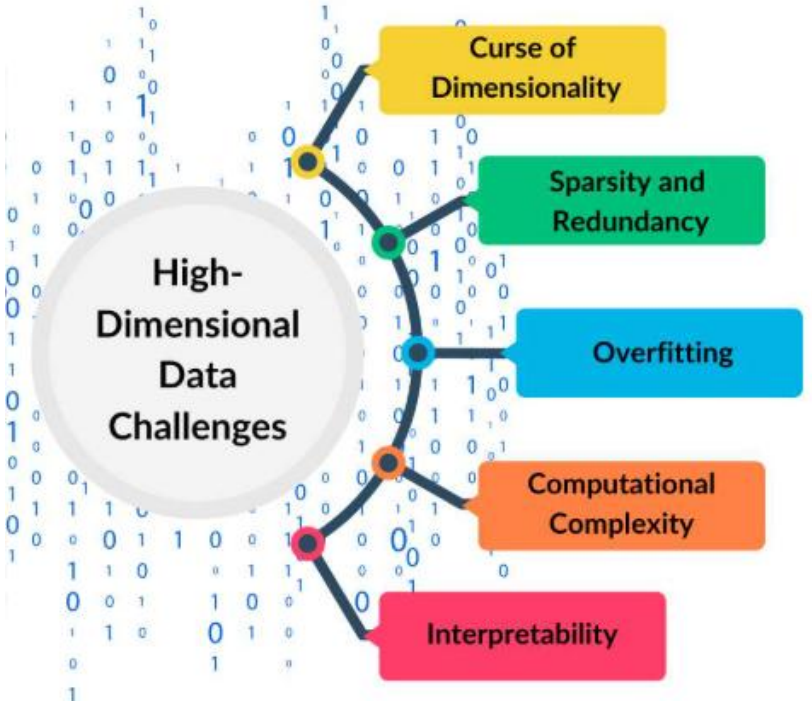
Ambiguity and Nuances: Words and phrases can have multiple meanings depending on the context, making them difficult to interpret.

Variability: Text varies widely in length, structure, and style, requiring extensive cleaning and standardization.

Noisy Data: Text often contains irrelevant information like punctuation, common words, and HTML tags that must be removed.

High Dimensionality: A large vocabulary creates a massive number of features, increasing

- **computational complexity** : Increased resources needed for processing and analysis.
- **risk of overfitting**: Machine learning models may fit noise rather than underlying patterns.
- **curse of dimensionality:** data becomes too sparse to find meaningful patterns.



Key Reasons for Text Processing

Given the unstructured and complex nature of text data, **preprocessing is essential for transforming raw text into a structured and analyzable format** suitable for machine learning models:

Key Reasons for Text Processing

Given the unstructured and complex nature of text data, **preprocessing is essential for transforming raw text into a structured and analyzable format** suitable for machine learning models:

Noise Reduction: Eliminating irrelevant or redundant information and improves the performance of machine learning models by focusing on meaningful data.

Key Reasons for Text Processing

Given the unstructured and complex nature of text data, **preprocessing is essential for transforming raw text into a structured and analyzable format** suitable for machine learning models:

Noise Reduction: Eliminating irrelevant or redundant information and improves the performance of machine learning models by focusing on meaningful data.

Standardization: Converting text to a standardized format that helps in reducing variability and enhancing the reliability of the analysis."

Key Reasons for Text Processing

Given the unstructured and complex nature of text data, **preprocessing is essential for transforming raw text into a structured and analyzable format** suitable for machine learning models:

Noise Reduction: Eliminating irrelevant or redundant information and improves the performance of machine learning models by focusing on meaningful data.

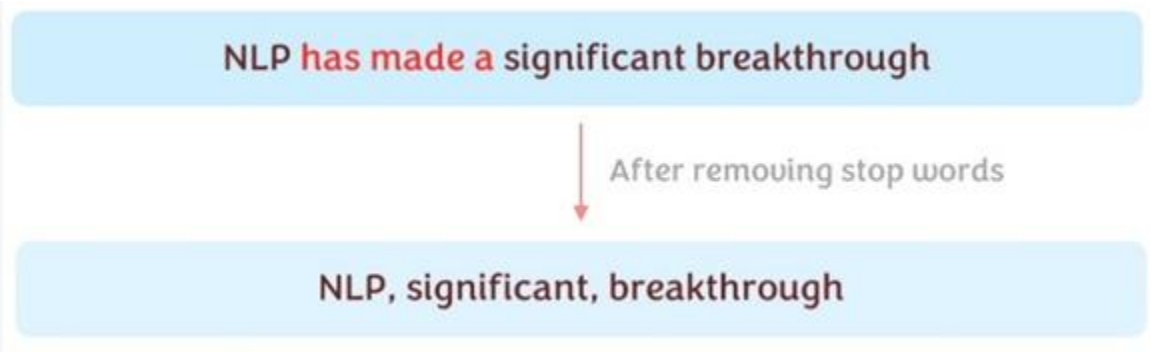
Standardization: Converting text to a standardized format that helps in reducing variability and enhancing the reliability of the analysis."

Feature Extraction: This is an essential part of preprocessing that involves techniques such as tokenization, vectorization, and embedding representations. These features are then used by machine learning models to learn patterns and make predictions or classifications.

Noise Reduction

1. Punctuation Removal: Punctuation marks such as commas, periods, question marks, and other symbols often do not carry significant meaning in text analysis. Removing these elements can help simplify the text and reduce noise.

Noise Reduction



1. Punctuation Removal: Punctuation marks such as commas, periods, question marks, and other symbols often do not carry significant meaning in text analysis. Removing these elements can help simplify the text and reduce noise.
2. Stop Word Removal: Stop words are common words such as 'and,' 'the,' 'is,' and 'in,' which do not contribute much to the meaning of a sentence. Eliminating these words helps to focus on the more meaningful words that are essential for analysis.

Noise Reduction

1. Punctuation Removal: Punctuation marks such as commas, periods, question marks, and other symbols often do not carry significant meaning in text analysis. Removing these elements can help simplify the text and reduce noise.
2. Stop Word Removal: Stop words are common words such as 'and,' 'the,' 'is,' and 'in,' which do not contribute much to the meaning of a sentence. Eliminating these words helps to focus on the more meaningful words that are essential for analysis.
3. Non-essential Elements: This includes removing numbers, special characters, HTML tags, or any other elements that do not add value to the understanding of the text.

Standardization

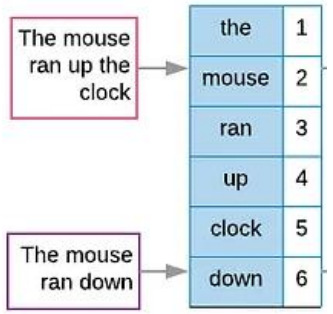
1. Lowercasing: This step involves converting all the letters in a text to lowercase. The main purpose of lowercasing is to ensure that words like 'Apple' and 'apple' are not treated as different entities by the system, thus avoiding any discrepancies caused by capitalization.

2. Stemming: Stemming is the process of reducing words to their base or root form. For example, the word 'running' can be reduced to the root form 'run' This helps in treating different morphological variants of a word as a single term.

3. Lemmatization: "Lemmatization is a process similar to stemming, but it is more sophisticated and context-aware. It reduces words to their dictionary or canonical form. For instance, "better" is lemmatized to "good."

Feature Extraction

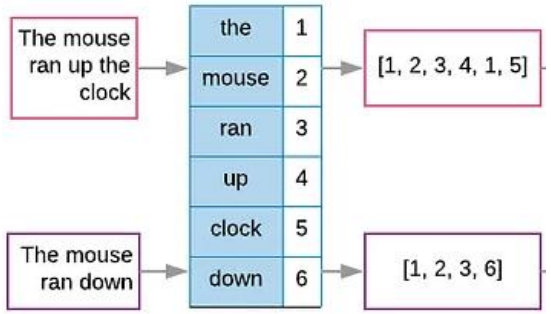
1. Tokenization: This is an essential process involving breaking down the text into individual units called tokens, which can be as small as words or as large as phrases. It organizes text into manageable and structured pieces.



Feature Extraction

1. Tokenization: This is an essential process involving breaking down the text into individual units called tokens, which can be as small as words or as large as phrases. It organizes text into manageable and structured pieces.

2. Vectorization: After the text has been tokenized, the next step is vectorization, where these tokens are converted into numerical vectors. Techniques include Bag of Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF).

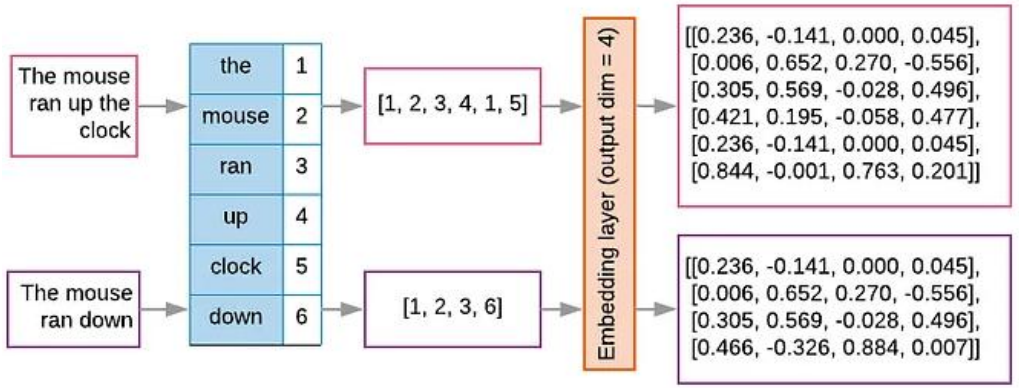


Feature Extraction

1. Tokenization: This is an essential process involving breaking down the text into individual units called tokens, which can be as small as words or as large as phrases. It organizes text into manageable and structured pieces.

2. Vectorization: After the text has been tokenized, the next step is vectorization, where these tokens are converted into numerical vectors. Techniques include Bag of Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF).

3. Embedding Representations: Embedding represents a more advanced technique in natural language processing, where words or phrases are mapped to high-dimensional vectors. Popular methods like Word2Vec, GloVe, and BERT capture intricate semantic relationships between words.

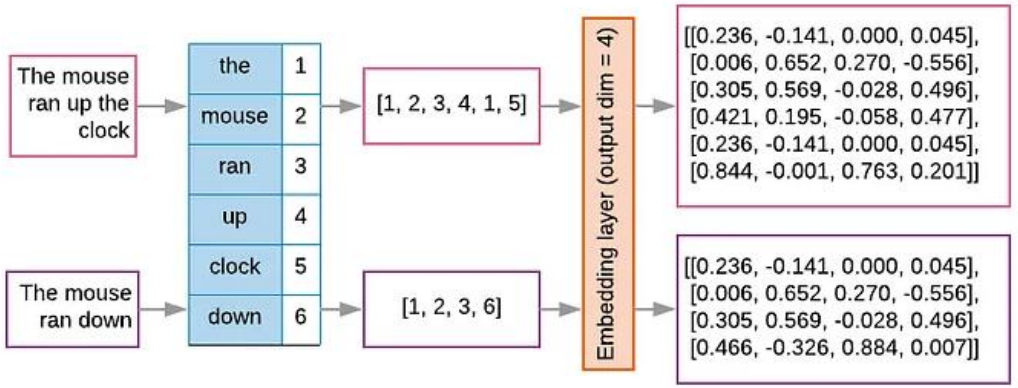


Feature Extraction

1. **Tokenization**: This is an essential process involving breaking down the text into individual units called tokens, which can be as small as words or as large as phrases. It organizes text into manageable and structured pieces.

2. **Vectorization**: After the text has been tokenized, the next step is vectorization, where these tokens are converted into numerical vectors. Techniques include Bag of Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF).

3. **Embedding Representations**: Embedding represents a more advanced technique in natural language processing, where words or phrases are mapped to high-dimensional vectors. Popular methods like Word2Vec, GloVe, and BERT capture intricate semantic relationships between words.



Parameters	Tokenization	Embedding
Definition	Converts large text into separate words, subwords, or characters (tokens).	Maps tokens into dense, continuous vector representations.
Use	Pre-processing text into manageable units.	Capturing semantic meaning so models can analyze and interpret.
Output	Sequence of tokens with index values.	Sequence of fixed-size vector representations.
Example	`"Machine Learning"` → `["Machine", "Learning"]`	`["Machine", "Learning"]` → `[embedding₁, embedding₂]`
Granularity	Character-level → very fine; word-level → coarser; sub-word → intermediate.	Higher granularity → more detailed vectors; lower → more abstract.
Tool Stack	Tokenizers: Byte Pair Encoding, SentencePiece, WordPiece.	Embeddings: GloVe, BERT, Word2Vec, DeBERTa.
Common Libraries	`spaCy`, `nltk`, `transformers`.	`torch.nn.Embedding`, `gensim`, `transformers`.

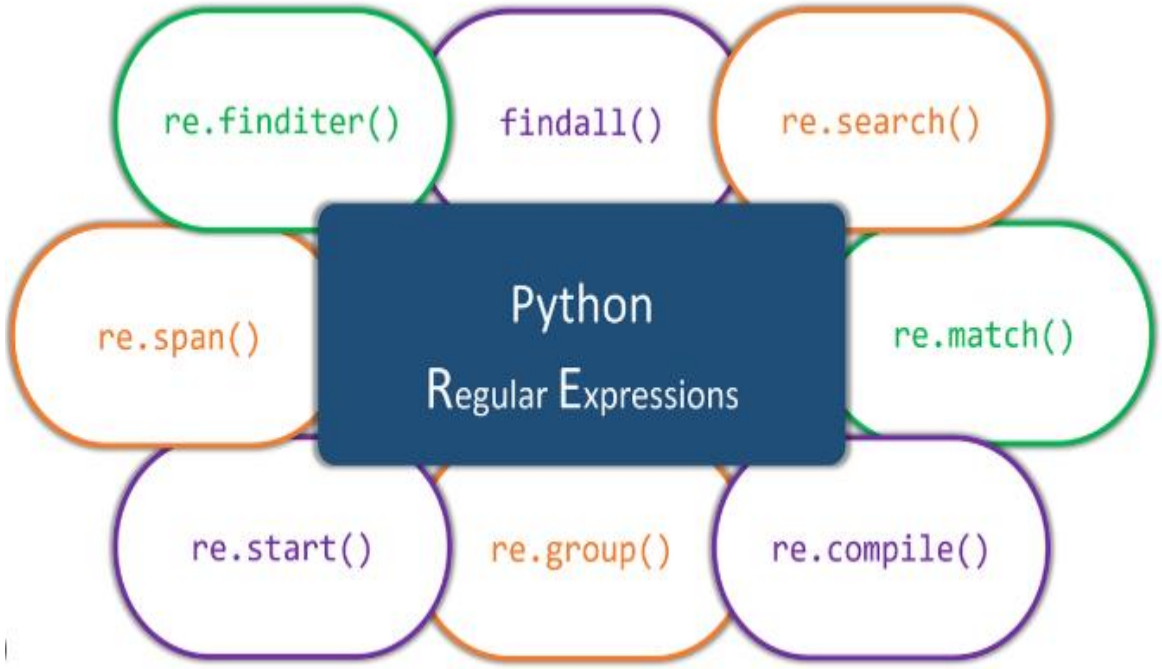
Regular Expression (Regex)

- A sequence of characters that defines a search pattern.
- A concise and flexible tool for finding, replacing, and manipulating text.
- Used to match patterns within strings, rather than fixed text.
- Often used in programming languages, text editors, and search engines for tasks like data validation, parsing, and web scraping.

The re Module in Python

Python integrates regular expressions through its built-in re module. This module provides various functions for searching, matching, and manipulating strings based on predefined patterns. Key functions include:

- re.search(): Used to find the first occurrence of a pattern in a string.
- re.match(): Checks for a match only at the beginning of the string (not explicitly detailed in provided excerpts but generally part of the module).
- re.findall(): Returns all non-overlapping matches of a pattern in a string as a list of strings.
- re.sub(): Replaces occurrences of a pattern in a string with a specified replacement



Basic Components of Regex

Regular Expression Basics	
.	Any character except newline
a	The character a
ab	The string ab
a|b	a or b
a*	0 or more a's
\	Escapes a special character

Regular Expression Quantifiers	
*	0 or more
+	1 or more
?	0 or 1
{2}	Exactly 2
{2, 5}	Between 2 and 5
{2,}	2 or more
{,5}	Up to 5

Default is greedy. Append ? for reluctant.

Regular Expression Groups	
(...)	Capturing group
(?P<Y>...)	Capturing group named Y
(?:...)	Non-capturing group
\Y	Match the Y'th captured group
(?P=Y)	Match the named group Y
(?#...)	Comment

Regular Expression Character Classes	
[ab-d]	One character of: a, b, c, d
[^ab-d]	One character except: a, b, c, d
[\b]	Backspace character
\d	One digit
\D	One non-digit
\s	One whitespace
\S	One non-whitespace
\w	One word character
\W	One non-word character

Regular Expression Assertions	
^	Start of string
\A	Start of string, ignores m flag
$	End of string
\Z	End of string, ignores m flag
\b	Word boundary
\B	Non-word boundary
(?=...)	Positive lookahead
(?!...)	Negative lookahead
(?<=...)	Positive lookbehind
(?<!...)	Negative lookbehind
(?()|)	Conditional

Regular Expression Flags	
i	Ignore case
m	^ and $ match start and end of line
s	. matches newline as well
x	Allow spaces and comments
L	Locale character classes
u	Unicode character classes
(?iLmsux)	Set flags within regex

Regular Expression Special Characters	
\n	Newline
\r	Carriage return
\t	Tab
\YYY	Octal character YYY
\xYY	Hexadecimal character YY

Regular Expression Replacement	
\g<0>	Insert entire match
\g<Y>	Insert match Y (name or number)
\Y	Insert group numbered Y

Practical Application of Regex

Regular expressions are incredibly versatile and can be used for a wide range of tasks, from simple search and replace operations to complex text extraction and validation.

- Text Search: Finding and locating specific words, phrases, or patterns within a larger body of text.
- Data Validation: Verifying that a string conforms to a specific format, such as an email address or a phone number.
- Text Processing & Extraction: Removing or extracting specific parts of a string based on a pattern, often for data cleaning (e.g., removing HTML tags, extracting hashtags).
- Data Transformation: Modifying the structure or format of text data, like standardizing phone numbers or converting all text to lowercase.

Basic pattern matching using *search()*

```
import re

pattern = "Python"
text = "I write code in Python."
match = re.search(pattern, text)
print("Match found:", bool(match))
```

Using *findall()* to find all occurrences of a pattern

```
import re

pattern = "a"
text = "I love to code in Python and it's amazing!"
matches = re.findall(pattern, text)
print("Matches found:", len(matches))
```

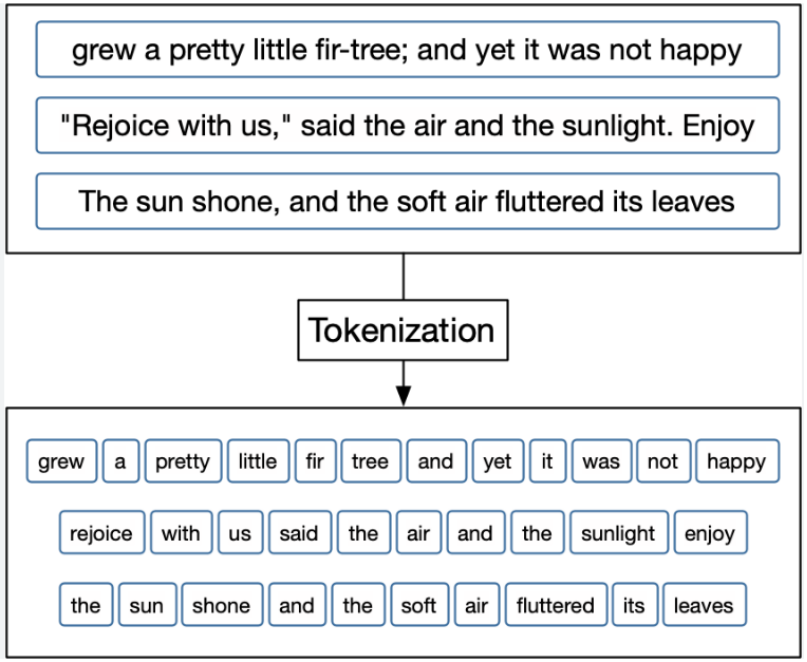
Using *sub()* to replace a pattern with a string

```
import re

pattern = "Python"
replacement = "Java"
text = "Python is fun"
substituted_text = re.sub(pattern, replacement, text)
print("Substituted text:", substituted_text)
```

Important Tokenization

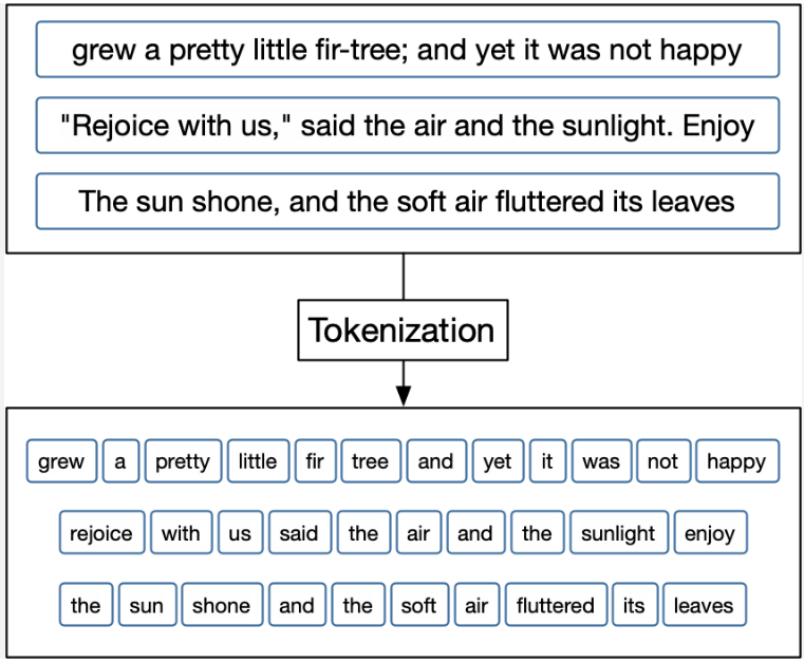
tokenization is a fundamental and indispensable step in the text preprocessing pipeline for Natural Language Processing. It involves breaking down a piece of text into smaller units called tokens. The core purpose is to convert unstructured text into a structured format that can be easily analyzed and processed by algorithms



Important Tokenization

tokenization is a fundamental and indispensable step in the text preprocessing pipeline for Natural Language Processing. It involves breaking down a piece of text into smaller units called tokens. The core purpose is to convert unstructured text into a structured format that can be easily analyzed and processed by algorithms

Simplification: It breaks down complex text into smaller, manageable units, typically words or phrases, making analysis more efficient and straightforward" and preventing it from being overwhelming"

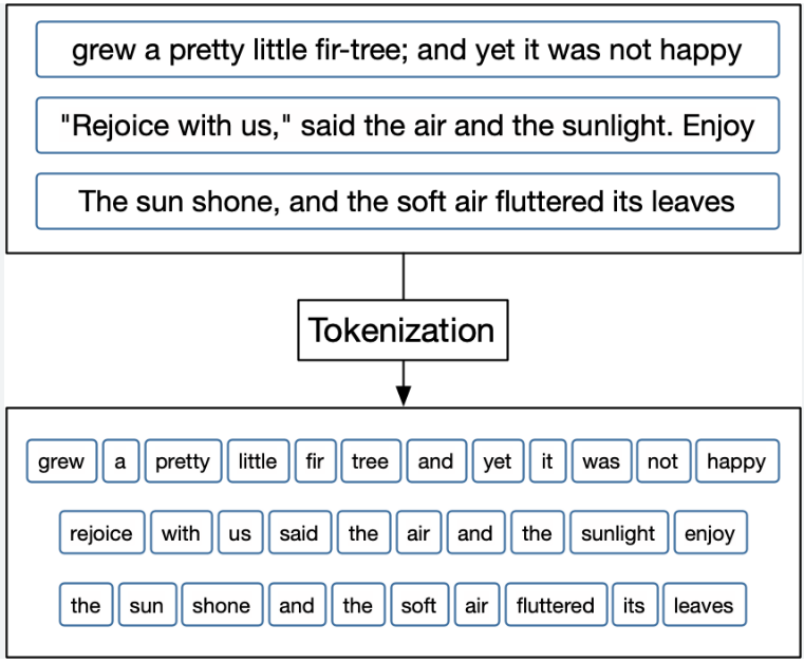


Important Tokenization

tokenization is a fundamental and indispensable step in the text preprocessing pipeline for Natural Language Processing. It involves breaking down a piece of text into smaller units called tokens. The core purpose is to convert unstructured text into a structured format that can be easily analyzed and processed by algorithms

Simplification: It breaks down complex text into smaller, manageable units, typically words or phrases, making analysis more efficient and straightforward" and preventing it from being overwhelming"

Standardization: Tokenization creates a consistent and uniform representation of the text, which is essential for subsequent processing and analysis because it ensures that the text is in a predictable format" and helps manage "inconsistencies and errors



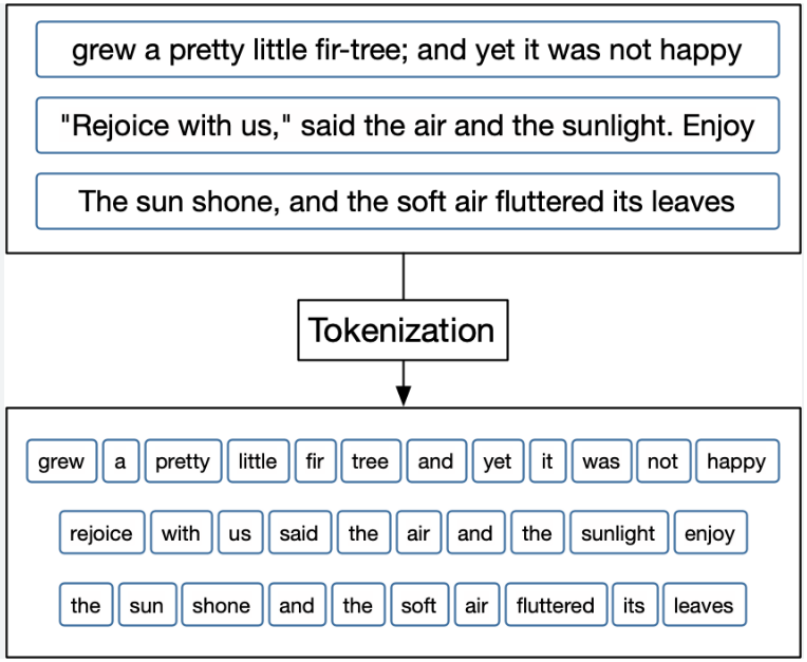
Important Tokenization

tokenization is a fundamental and indispensable step in the text preprocessing pipeline for Natural Language Processing. It involves breaking down a piece of text into smaller units called tokens. The core purpose is to convert unstructured text into a structured format that can be easily analyzed and processed by algorithms

Simplification: It breaks down complex text into smaller, manageable units, typically words or phrases, making analysis more efficient and straightforward" and preventing it from being overwhelming"

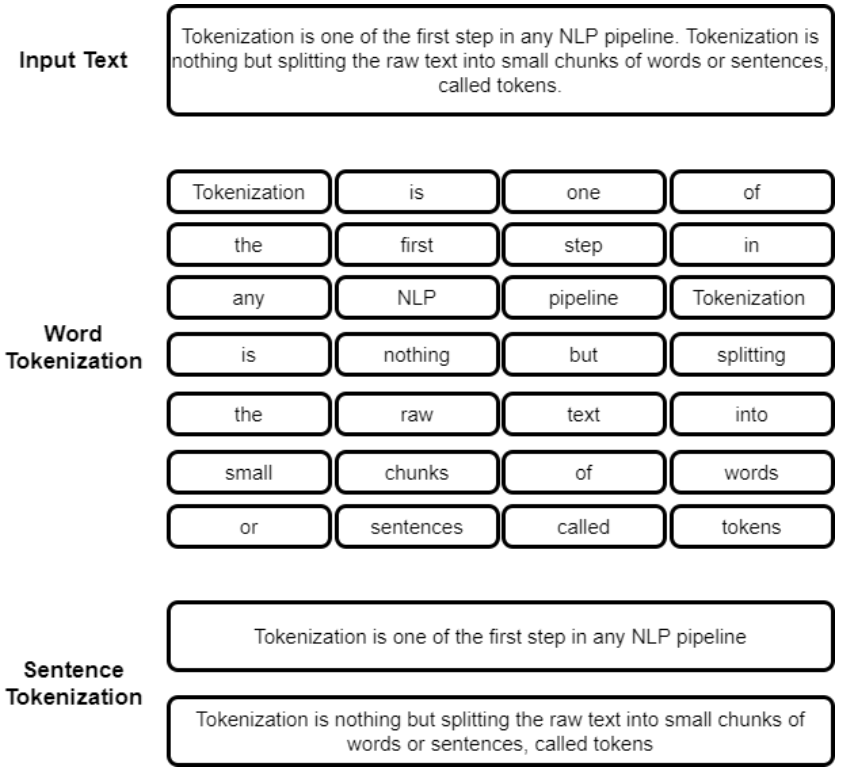
Standardization: Tokenization creates a consistent and uniform representation of the text, which is essential for subsequent processing and analysis because it ensures that the text is in a predictable format" and helps manage "inconsistencies and errors

Feature Extraction: It facilitates the extraction of meaningful features from the text. By breaking text into tokens, it becomes possible to identify and utilize these features as inputs in various machine learning models, enabling tasks like sentiment analysis, and execute various other natural language processing tasks



Type of Tokenization

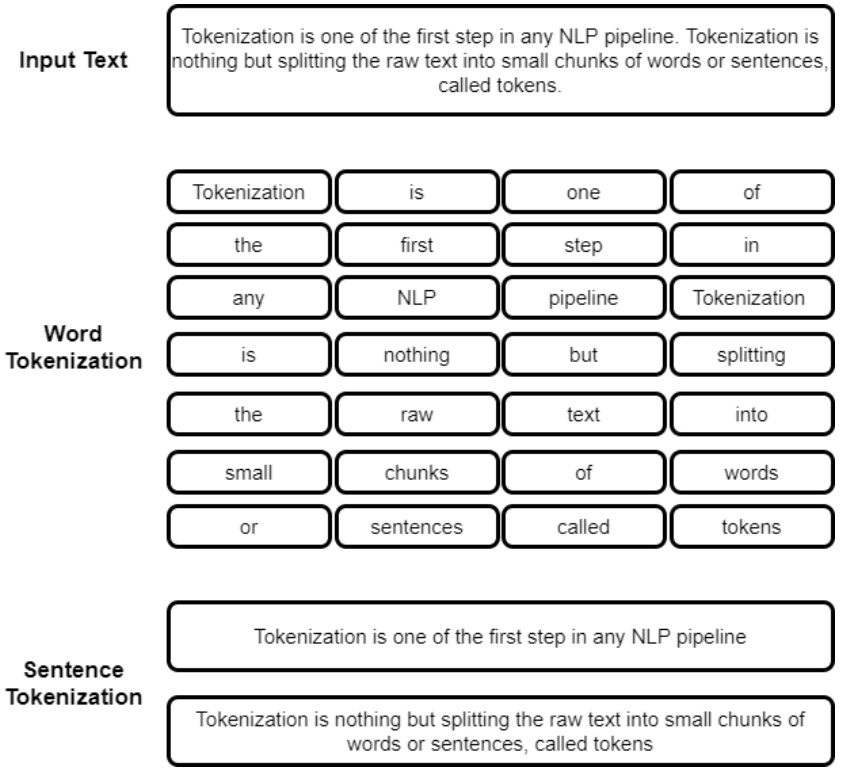
Word Tokenization: The most common type of tokenization, it splits text into individual words. This is foundational for tasks like text classification, sentiment analysis, and named entity recognition. Libraries like NLTK and spaCy offer robust functions for this.



Type of Tokenization

Word Tokenization: The most common type of tokenization, it splits text into individual words. This is foundational for tasks like text classification, sentiment analysis, and named entity recognition. Libraries like NLTK and spaCy offer robust functions for this.

Sentence Tokenization: Breaks text down into individual sentences. This is crucial for summarization and sentiment analysis at a sentence level, helping to preserve the text's structure and meaning.

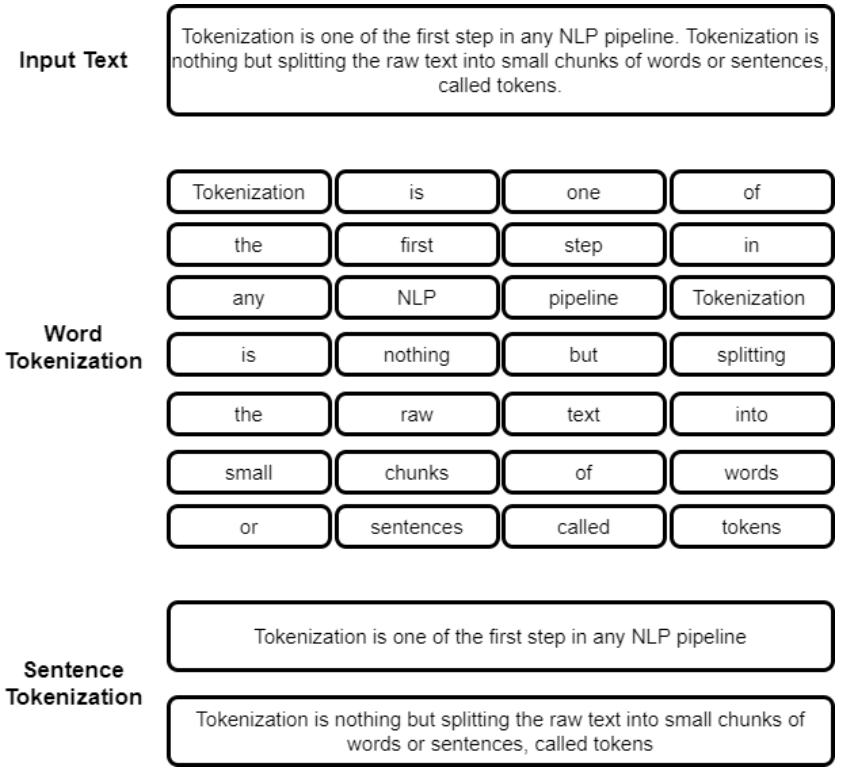


Type of Tokenization

Word Tokenization: The most common type of tokenization, it splits text into individual words. This is foundational for tasks like text classification, sentiment analysis, and named entity recognition. Libraries like NLTK and spaCy offer robust functions for this.

Sentence Tokenization: Breaks text down into individual sentences. This is crucial for summarization and sentiment analysis at a sentence level, helping to preserve the text's structure and meaning.

Character Tokenization: Splits text into single characters. This is useful for tasks requiring fine-grained analysis, such as language modeling, handwriting recognition, and spell-checking.



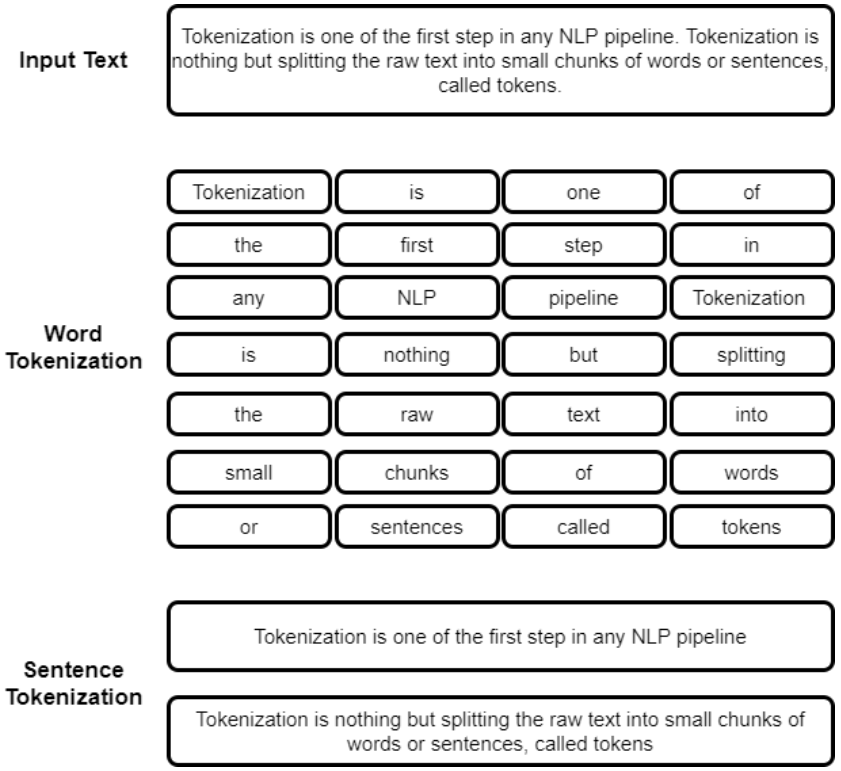
Type of Tokenization

Word Tokenization: The most common type of tokenization, it splits text into individual words. This is foundational for tasks like text classification, sentiment analysis, and named entity recognition. Libraries like NLTK and spaCy offer robust functions for this.

Sentence Tokenization: Breaks text down into individual sentences. This is crucial for summarization and sentiment analysis at a sentence level, helping to preserve the text's structure and meaning.

Character Tokenization: Splits text into single characters. This is useful for tasks requiring fine-grained analysis, such as language modeling, handwriting recognition, and spell-checking.

Alphanumeric Word Tokenization: A specific word tokenization technique that typically involves removing punctuation and other non-alphanumeric characters to convert unstructured text into a structured, cleaner format for algorithms.



Tokenization using Python Libraries

Word Tokenization

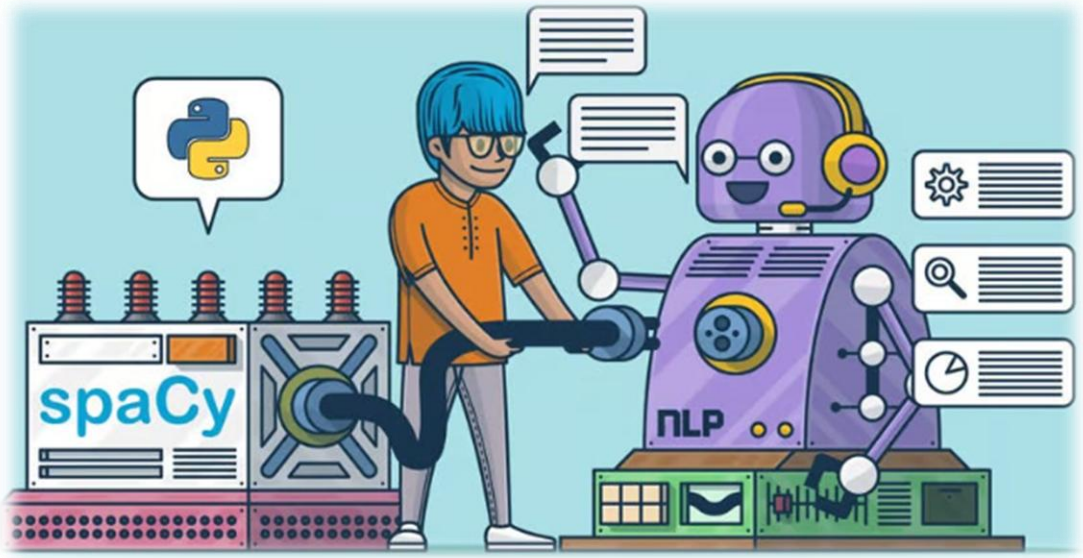
- **NLTK:** Uses the `word_tokenize` function to split text into words. You need to download the `punkt` model first.
  - **Example:** `nltk.word_tokenize("Hello world.")` → `['Hello', 'world', '.']`
- **SpaCy:** Processes text into a `Doc` object and then extracts tokens. It's often preferred for production due to its speed and efficiency.
  - **Example:** `doc = nlp("Hello world.")` → `[token.text for token in doc]` → `['Hello', 'world', '.']`

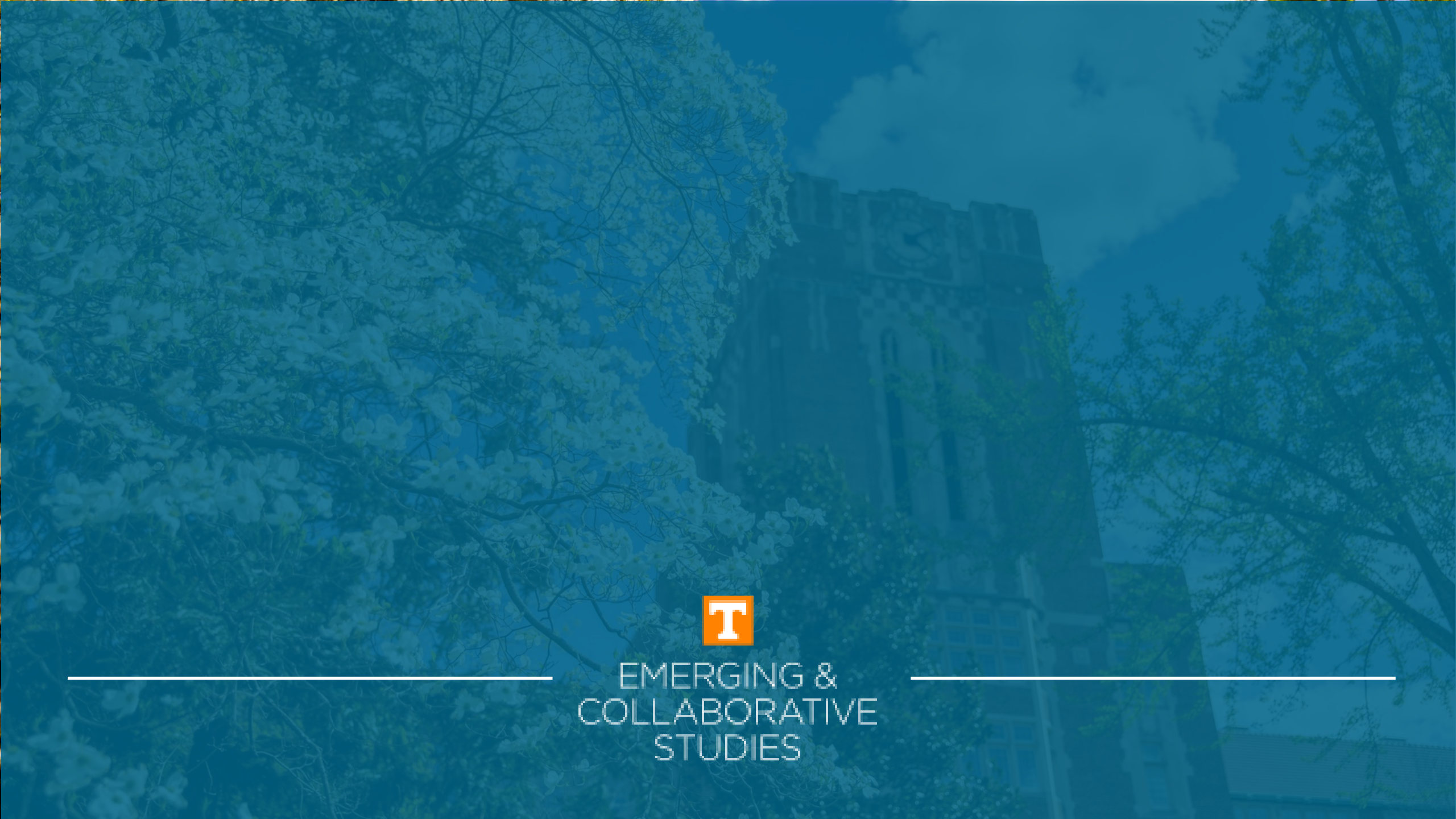
Sentence Tokenization

- **NLTK:** Employs the `sent_tokenize` function to split a document into sentences. It's a useful heuristic for structured text.
  - **Example:** `nltk.sent_tokenize("Hello world. This is a test.")` → `['Hello world.', 'This is a test.']`
- **SpaCy:** After creating a `Doc` object, you can access sentences via `doc.sents`.
  - **Example:** `doc = nlp("Hello world. This is a test.")` → `[sent.text for sent in doc.sents]` → `['Hello world.', 'This is a test.']`

Character Tokenization

- This is the simplest form, where a string is converted into a list of its individual characters.
  - **Example:** `list("Hello")` → `['H', 'e', 'l', 'l', 'o']`




EMERGING &
COLLABORATIVE
STUDIES